

HO 20: Implementazione di Raft in C

Panoramica del codice e istruzioni per l'uso

Last Update: 21 maggio 2026

Panoramica

Obiettivo dell'HO è la **comprensione** dell'algoritmo RAFT e della implementazione proposta in C, attraverso la scrittura di un report scientifico.

Il file `raft_impl.c` implementa il protocollo Raft su un cluster di $N = 5$ server che girano in locale come processi separati e comunicano tramite socket UDP. Un processo padre funge da client interattivo.

Le scelte tecniche principali sono tre. Si usa **UDP** perché Raft è già progettato per tollerare messaggi persi: i timeout di elezione e i re-invi del leader gestiscono i casi di perdita senza bisogno di connessioni TCP. Si usa `fork()` per creare un processo indipendente per ogni server, in modo che ogni processo abbia il proprio stato e possa essere killato indipendentemente per simulare un crash. Si usa `select()` con timeout per gestire la concorrenza all'interno di ogni processo: un singolo loop aspetta messaggi sul socket oppure l'evento temporale (scadenza del timeout di elezione o heartbeat), senza bloccarsi.

Struttura del codice

Il codice è organizzato in quattro parti logiche.

Strutture dati

`entry_t` rappresenta una entry del log: il *term* in cui è stata creata e il comando (un intero). `srv_t` contiene lo stato completo di un server: il term corrente, per chi ha votato, il log, il commit index, e — se il server è leader — gli array `next_idx[]` e `match_idx[]` che tracciano lo stato di replicazione su ogni follower.

I messaggi sono definiti come strutture separate (`msg_rv_t`, `msg_ae_t`, ecc.) raccolte in una union `msg_t` che permette di leggere qualsiasi messaggio in un unico buffer senza copie.

Utilità

`ms_now()` restituisce i millisecondi monotoni correnti, usato per calcolare i timeout. `send_to()` e `broadcast()` incapsulano `sendto()` su UDP. `at_least_uptodate()` implementa il confronto del log per l'elezione (§5.4.1 del paper): prima si confronta il term dell'ultima entry, poi — a parità di term — la lunghezza del log.

Transizioni di stato e replicazione

Tre funzioni gestiscono le transizioni principali: `become_follower()`, `start_election()` e `become_leader()`. La funzione `send_ae()` costruisce e invia un messaggio `AppendEntries` a un singolo follower, includendo tutte le entry a partire da `next_idx[dest]`. `update_commit()` scorre il log all'indietro cercando il massimo indice replicato sulla maggioranza nel term corrente.

Handler dei messaggi e loop principale

Cinque funzioni `on_*`() gestiscono i messaggi in ingresso. Il loop principale in `server_loop()` chiama `select()` con un timeout calcolato dinamicamente: per il leader è il tempo mancante al prossimo heartbeat; per follower e candidate è il tempo mancante alla scadenza del timeout di elezione. Se arriva un messaggio lo dispatcha all'handler corretto; se scade il timeout, il leader invia heartbeat o il follower avvia un'elezione.

Compilazione ed esecuzione

Compilazione

```
gcc -Wall -Wextra -o raft raft_impl.c
```

Non sono necessarie librerie esterne. Il codice usa solo API POSIX standard disponibili su qualsiasi sistema Linux.

Esecuzione

```
./raft
```

Il programma avvia 5 processi server (porte 5000–5004) e attende circa 300 ms per l'elezione del primo leader. Il processo padre diventa il client interattivo: digita un intero e premi Invio per inviare un comando al cluster. Il leader lo aggiungerà al log, lo replicherà sui follower e lo committerà non appena la maggioranza avrà confermato.

Output atteso al lancio:

```
=== Raft demo: 5 server, porte 5000-5004 ===

[S0] avviato sulla porta 5000
...
[S1] FOLLOWER -> CANDIDATE (term 1)
[S0] RequestVote da S1 (term 1): GRANTED
[S2] RequestVote da S1 (term 1): GRANTED
[S1] voto da S0 -- totale 2/5
[S1] voto da S2 -- totale 3/5
[S1] CANDIDATE -> LEADER (term 1)

[client] pronto. Digita un intero + Invio per inviare un comando.
```

Inviando un comando:

```
42
[client] comando 42 inviato
[S1] LEADER: aggiunto index 1 (cmd=42, term=1)
[S1] LEADER: committed index 1 (cmd=42)
[S0] follower: commitIndex aggiornato a 1
[S2] follower: commitIndex aggiornato a 1
[S3] follower: commitIndex aggiornato a 1
[S4] follower: commitIndex aggiornato a 1
```

Terminare il programma

Premi **Ctrl-C** oppure chiudi lo stdin con **Ctrl-D**. Per killare tutti i processi in modo pulito:

```
pkill raft
```

Simulare un crash

Identificare i processi

Apri un secondo terminale. Per vedere tutti i processi del cluster:

```
pgrep -a raft | sort
```

L'output mostra il padre (client) e i cinque figli. I figli vengono creati in ordine nel ciclo `for (i=0; i<N; i++) fork()`, quindi il primo figlio nell'elenco ordinato per PID è S0, il secondo è S1, e così via.

Dall'output del programma sai già quale server è il leader (cerca la riga `CANDIDATE -> LEADER`). Il PID corrispondente è il $(leader + 1)$ -esimo figlio nell'elenco ordinato.

Crash del leader

```
kill -9 <PID del leader>
```

Nel terminale principale vedrai gli altri server accorgersi della mancanza di heartbeat entro 150–300 ms e avviare una nuova elezione al term successivo.

```
[S2] FOLLOWER -> CANDIDATE (term 2)
[S0] RequestVote da S2 (term 2): GRANTED
[S1] RequestVote da S2 (term 2): GRANTED
[S2] voto da S0 -- totale 2/5
[S2] voto da S1 -- totale 3/5
[S2] CANDIDATE -> LEADER (term 2)
```

Crash di un follower

Killa un PID che non è il leader. Il cluster continua a funzionare perché 4 server su 5 sono ancora una maggioranza. I comandi vengono comunque committed normalmente.

[!] Nessuna persistenza su disco

Questa implementazione non salva lo stato su disco. Se un processo viene killato e poi riavviato (lanciando di nuovo `./raft`), ripartirà con log e term azzerati — un'approssimazione del crash accettabile per scopi didattici, ma non corretta in produzione. In un sistema reale `currentTerm`, `votedFor` e `log[]` andrebbero scritti su disco con `fsync()` prima di rispondere a qualsiasi messaggio.

Semplificazioni rispetto alla specifica completa

Funzionalità	Specifica Raft	Questa impl.
Persistenza	fsync prima di rispondere	non implementata
Snapshot	log compattato su disco	log in memoria, illimitato
Redirect client	follower reindirizza	follower ignora il comando
Ottimiz. nextIndex	ricerca binaria	arretramento lineare

Codice sorgente: raft_impl.c

```
1  /* raft_impl.c      implementazione didattica del consenso Raft
2  *
3  * Topologia: N processi su localhost, UDP, porte BASE_PORT..BASE_PORT+N
4  *             -1.
5  * Il processo padre (main) funge da client interattivo.
6  *
7  * Compilazione: gcc -Wall -Wextra -o raft raft_impl.c
8  * Esecuzione: ./raft
9  *             Digita interi + Invio per inviare comandi al cluster.
10 *             Ctrl-C per uscire.
11 */
12 #include <stdio.h>
13 #include <stdlib.h>
14 #include <string.h>
15 #include <unistd.h>
16 #include <time.h>
17 #include <errno.h>
18 #include <sys/socket.h>
19 #include <sys/select.h>
20 #include <sys/wait.h>
21 #include <netinet/in.h>
22 #include <arpa/inet.h>
23
24 /*      Costanti
25 */
26 #define N          5          /* numero di server nel cluster */
27 #define BASE_PORT  5000      /* server i ascolta su BASE_PORT+i */
28 #define MAX_LOG    256      /* lunghezza massima del log (entry) */
29 #define HB_MS      50        /* heartbeat ogni HB_MS millisecondi */
30 #define EMIN_MS    150       /* timeout elezione: minimo in ms */
31 #define EMAX_MS    300       /* timeout elezione: massimo in ms (random) */
32 #define AE_BATCH   8         /* entry max per messaggio AppendEntries */
33
34 /*      Tipi di messaggio
35 */
36 #define MSG_RV      1         /* RequestVote          candidate      tutti */
37 #define MSG_RVR     2         /* RequestVote Reply   tutti          candidate */
38 #define MSG_AE      3         /* AppendEntries       leader         follower */
39 #define MSG_AER     4         /* AppendEntries Reply follower        leader */
40 #define MSG_CLIENT  5         /* Comando client      client          tutti */
41
42 /*      Stato del server
43 */
44 typedef enum { FOLLOWER, CANDIDATE, LEADER } role_t;
45 static const char *ROLE[] = { "FOLLOWER", "CANDIDATE", "LEADER" };
46
47 /* Un'entry del log: il term in cui stata creata + il comando */
48 typedef struct { int term; int cmd; } entry_t;
49
50 /* Header comune a tutti i messaggi */
51 typedef struct { int type; int term; int from; } hdr_t;
52
53 /*      Strutture dei messaggi
54 */
55
56 /* RequestVote: il candidato dichiara il proprio log */
```

```

53 typedef struct {
54     hdr_t h;
55     int last_log_idx; /* indice ultima entry nel log del candidato */
56     int last_log_term; /* term ultima entry nel log del candidato */
57 } msg_rv_t;
58
59 /* RequestVote Reply */
60 typedef struct {
61     hdr_t h;
62     int granted; /* 1 = voto concesso, 0 = negato */
63 } msg_rvr_t;
64
65 /* AppendEntries (anche heartbeat se n==0) */
66 typedef struct {
67     hdr_t h;
68     int prev_idx; /* indice entry immediatamente precedente */
69     int prev_term; /* term entry immediatamente precedente */
70     int commit; /* commitIndex del leader */
71     int n; /* numero di entry nel messaggio */
72     entry_t entries[AE_BATCH];
73 } msg_ae_t;
74
75 /* AppendEntries Reply */
76 typedef struct {
77     hdr_t h;
78     int success; /* 1 = log consistente, entry aggiunte */
79     int match; /* massimo indice replicato con successo */
80 } msg_aer_t;
81
82 /* Comando client */
83 typedef struct {
84     hdr_t h;
85     int cmd;
86 } msg_client_t;
87
88 /* Union: permette di leggere qualsiasi messaggio in un buffer unico */
89 typedef union {
90     hdr_t h;
91     msg_rv_t rv;
92     msg_rvr_t rvr;
93     msg_ae_t ae;
94     msg_aer_t aer;
95     msg_client_t client;
96 } msg_t;
97
98 /* Stato completo di un server
99
100 */
101 typedef struct {
102     /* identit */
103     int id;
104     role_t role;
105
106     /* stato persistente (in un sistema reale: su disco) */
107     int term; /* currentTerm: il term pi alto visto */
108     int voted_for; /* -1 = non votato in questo term */
109     entry_t log[MAX_LOG];
110     int log_len; /* numero di entry nel log (indici 1-based) */

```

```

110     /* stato volatile */
111     int    commit_idx;      /* indice ultima entry committed      */
112     int    votes;          /* voti raccolti come candidate      */
113
114     /* stato volatile del leader (si azzera a ogni elezione) */
115     int    next_idx[N];    /* prossima entry da inviare a ogni follower */
116     int    match_idx[N];   /* ultima entry confermata da ogni follower */
117
118     /* timing */
119     long   hb_recv_at;     /* timestamp ultimo heartbeat ricevuto (ms) */
120     long   hb_sent_at;     /* timestamp ultimo heartbeat inviato (ms) */
121     int    elect_ms;       /* timeout elezione (scelto casualmente) */
122
123     /* socket UDP */
124     int    fd;
125 } srv_t;
126
127 /*
128
129
130
131  * Utilit 
132
133
134
135  */
136
137  /* Millisecondi monotoni dall'avvio del sistema */
138  static long ms_now(void) {
139      struct timespec t;
140      clock_gettime(CLOCK_MONOTONIC, &t);
141      return t.tv_sec * 1000L + t.tv_nsec / 1000000L;
142  }
143
144  /* Invia buf su UDP alla porta indicata (localhost) */
145  static void udp_send(int fd, int port, const void *buf, size_t len) {
146      struct sockaddr_in a = {
147          .sin_family    = AF_INET,
148          .sin_port      = htons(port),
149          .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
150      };
151      sendto(fd, buf, len, 0, (struct sockaddr *)&a, sizeof(a));
152  }
153
154  static void send_to(srv_t *s, int dest, const void *m, size_t len) {
155      udp_send(s->fd, BASE_PORT + dest, m, len);
156  }
157
158  static void broadcast(srv_t *s, const void *m, size_t len) {
159      for (int i = 0; i < N; i++)
160          if (i != s->id)
161              send_to(s, i, m, len);
162  }
163
164  /* Restituisce il term dell'entry in posizione idx (1-based); 0 se fuori
165     range */
166  static int log_term_at(srv_t *s, int idx) {
167      return (idx > 0 && idx <= s->log_len) ? s->log[idx-1].term : 0;
168  }
169
170  /* 5.4.1: il log "loro" almeno aggiornato quanto "il mio"?
171     * Prima si confronta il term dell'ultima entry, poi la lunghezza. */
172  static int at_least_uptodate(int my_term, int my_idx,
173                              int their_term, int their_idx) {
174      if (their_term != my_term) return their_term > my_term;
175  }

```

```

168     return their_idx >= my_idx;
169 }
170
171 /* Sceglie un nuovo timeout di elezione casuale e azzerava il timer */
172 static void reset_election_timer(srv_t *s) {
173     s->hb_recv_at = ms_now();
174     s->elect_ms    = EMIN_MS + rand() % (EMAX_MS - EMIN_MS);
175 }
176
177 /*
178  * Transizioni di stato
179  *
180  */
181 static void become_follower(srv_t *s, int new_term) {
182     if (s->role != FOLLOWER)
183         printf("[S%d] %s FOLLOWER (term %d)\n",
184             s->id, ROLE[s->role], new_term);
185     else if (s->term != new_term)
186         printf("[S%d] term aggiornato: %d %d\n",
187             s->id, s->term, new_term);
188     s->role = FOLLOWER;
189     s->term = new_term;
190     s->voted_for = -1;
191     s->votes = 0;
192     reset_election_timer(s);
193 }
194
195 /* Scaduto il timeout di elezione: incrementa il term e chiedi voti */
196 static void start_election(srv_t *s) {
197     s->term++;
198     s->role = CANDIDATE;
199     s->voted_for = s->id; /* vota per s stesso */
200     s->votes = 1;
201     reset_election_timer(s);
202
203     printf("[S%d] FOLLOWER CANDIDATE (term %d)\n", s->id, s->term);
204
205     msg_rv_t m = {
206         .h = { MSG_RV, s->term, s->id },
207         .last_log_idx = s->log_len,
208         .last_log_term = log_term_at(s, s->log_len),
209     };
210     broadcast(s, &m, sizeof(m));
211 }
212
213 /* Voti sufficienti raccolti: diventa leader */
214 static void become_leader(srv_t *s) {
215     printf("[S%d] CANDIDATE LEADER (term %d)\n", s->id, s->term);
216     s->role = LEADER;
217
218     /* inizializza lo stato del leader per ogni follower */
219     for (int i = 0; i < N; i++) {
220         s->next_idx[i] = s->log_len + 1; /* ottimismo: il follower
221             aggiornato */
222         s->match_idx[i] = 0;
223     }
224     s->hb_sent_at = 0; /* forza heartbeat immediato */
225 }

```

```

226  /*

227  * Replicazione del log
228  *

        */

229
230  /* Invia AppendEntries (o heartbeat se il follower      aggiornato) */
231  static void send_ae(srv_t *s, int dest) {
232      msg_ae_t m;
233      m.h      = (hdr_t){ MSG_AE, s->term, s->id };
234      m.commit = s->commit_idx;
235
236      /* l'entry precedente serve al follower per verificare la consistenza
        */
237      int prev      = s->next_idx[dest] - 1;
238      m.prev_idx    = prev;
239      m.prev_term   = log_term_at(s, prev);
240
241      /* copia le entry da inviare (da next_idx[dest] in poi) */
242      int n = 0;
243      for (int i = s->next_idx[dest];
244           i <= s->log_len && n < AE_BATCH; i++, n++)
245          m.entries[n] = s->log[i-1];
246      m.n = n;
247
248      send_to(s, dest, &m, sizeof(m));
249  }
250
251  /* Invia heartbeat a tutti i follower */
252  static void send_heartbeats(srv_t *s) {
253      for (int i = 0; i < N; i++)
254          if (i != s->id)
255              send_ae(s, i);
256      s->hb_sent_at = ms_now();
257  }
258
259  /* Dopo aver ricevuto una risposta positiva: aggiorna commitIndex.
260   * 5.3: il leader fa commit di N se N      replicato sulla maggioranza
261   *      E log[N].term == currentTerm. */
262  static void update_commit(srv_t *s) {
263      for (int n = s->log_len; n > s->commit_idx; n--) {
264          /* solo entry del term corrente possono essere committed
            direttamente */
265          if (s->log[n-1].term != s->term) continue;
266          int cnt = 1; /* il leader stesso ha l'entry */
267          for (int i = 0; i < N; i++)
268              if (i != s->id && s->match_idx[i] >= n) cnt++;
269          if (cnt > N / 2) {
270              s->commit_idx = n;
271              printf("[S%d] LEADER: committed index %d (cmd=%d)\n",
272                     s->id, n, s->log[n-1].cmd);
273              break;
274          }
275      }
276  }
277
278  /*

279  * Handler dei messaggi in ingresso
280  *

```



```

281
282 static void on_request_vote(srv_t *s, msg_rv_t *m) {
283     /* concedi il voto se:
284      * 1. il term del candidato >= al nostro
285      * 2. non abbiamo gi votato per qualcun altro in questo term
286      * 3. il log del candidato almeno aggiornato quanto il nostro */
287     int grant = (m->h.term >= s->term) &&
288                 (s->voted_for == -1 || s->voted_for == m->h.from) &&
289                 at_least_uptodate(log_term_at(s, s->log_len), s->log_len,
290                                 m->last_log_term, m->last_log_idx);
291     if (grant) {
292         s->voted_for = m->h.from;
293         reset_election_timer(s); /* segnale di attivit : non candidarci
294                                 subito */
295     }
296     printf("[S%d] RequestVote da S%d (term %d): %s\n",
297           s->id, m->h.from, m->h.term, grant ? "GRANTED" : "DENIED");
298
299     msg_rvr_t r = { .h = { MSG_RVR, s->term, s->id }, .granted = grant };
300     send_to(s, m->h.from, &r, sizeof(r));
301 }
302
303 static void on_vote_response(srv_t *s, msg_rvr_t *m) {
304     /* ignora risposte obsolete o fuori ruolo */
305     if (s->role != CANDIDATE || m->h.term != s->term) return;
306     if (!m->granted) return;
307
308     s->votes++;
309     printf("[S%d] voto da S%d totale %d/%d\n",
310           s->id, m->h.from, s->votes, N);
311
312     /* maggioranza raggiunta: diventa leader */
313     if (s->votes > N / 2)
314         become_leader(s);
315 }
316
317 static void on_append_entries(srv_t *s, msg_ae_t *m) {
318     msg_aer_t r = { .h = { MSG_AER, s->term, s->id },
319                   .success = 0, .match = 0 };
320
321     /* messaggio da leader obsoleto: rifiuta e comunica il term corrente
322      */
323     if (m->h.term < s->term) {
324         send_to(s, m->h.from, &r, sizeof(r));
325         return;
326     }
327
328     /* leader valido: se eravamo candidate, torniamo follower */
329     if (s->role == CANDIDATE)
330         become_follower(s, m->h.term);
331
332     /* aggiorna il timer il leader vivo */
333     reset_election_timer(s);
334
335     /* 5 .3: controlla che il log sia consistente fino a prev_idx.
336      * Se prev_idx non esiste o ha term sbagliato, il follower indietro
337      */
338     if (m->prev_idx > 0 &&
339         (m->prev_idx > s->log_len ||
340          log_term_at(s, m->prev_idx) != m->prev_term)) {

```

```

339         send_to(s, m->h.from, &r, sizeof(r));
340         return;
341     }
342
343     /* scrivi le nuove entry nel log */
344     for (int i = 0; i < m->n; i++) {
345         int idx = m->prev_idx + 1 + i; /* 1-based */
346         /* se c'è un conflitto (term diverso), tronca il log da qui */
347         if (idx <= s->log_len && s->log[idx-1].term != m->entries[i].term)
348             s->log_len = idx - 1;
349         /* aggiungi l'entry se non già presente */
350         if (idx > s->log_len) {
351             s->log[s->log_len] = m->entries[i];
352             s->log_len++;
353         }
354     }
355
356     /* avanza commitIndex se il leader è avanti */
357     if (m->commit > s->commit_idx) {
358         s->commit_idx = (m->commit < s->log_len) ? m->commit : s->log_len;
359         printf("[S%d] follower: commitIndex aggiornato a %d\n",
360             s->id, s->commit_idx);
361     }
362
363     r.success = 1;
364     r.match = m->prev_idx + m->n; /* fino a qui il log è allineato */
365     send_to(s, m->h.from, &r, sizeof(r));
366 }
367
368 static void on_append_response(srv_t *s, msg_aer_t *m) {
369     if (s->role != LEADER || m->h.term != s->term) return;
370
371     int from = m->h.from;
372     if (m->success) {
373         /* il follower ha aggiunto le entry: aggiorna next/match */
374         if (m->match > s->match_idx[from]) {
375             s->match_idx[from] = m->match;
376             s->next_idx[from] = m->match + 1;
377         }
378         update_commit(s);
379     } else {
380         /* il follower non aveva l'entry prev: arretra e riprova */
381         if (s->next_idx[from] > 1)
382             s->next_idx[from]--;
383         send_ae(s, from);
384     }
385 }
386
387 static void on_client_cmd(srv_t *s, msg_client_t *m) {
388     /* solo il leader elabora i comandi; gli altri li ignorano */
389     if (s->role != LEADER) return;
390
391     /* aggiunge l'entry al log locale */
392     s->log[s->log_len++] = (entry_t){ .term = s->term, .cmd = m->cmd };
393     printf("[S%d] LEADER: aggiunto index %d (cmd=%d, term=%d)\n",
394         s->id, s->log_len, m->cmd, s->term);
395
396     /* avvia subito la replicazione */
397     for (int i = 0; i < N; i++)
398         if (i != s->id)
399             send_ae(s, i);
400 }
401

```

```

402  /*

403  * Loop principale del server
404  *

        */

405
406  static void server_loop(int id) {
407      setbuf(stdout, NULL); /* disabilita buffering: printf immediato */
408      /* seed diverso per ogni processo: timeout di elezione casuali */
409      srand((unsigned)(time(NULL) ^ (getpid() * 2654435761U)));
410
411      srv_t s;
412      memset(&s, 0, sizeof(s));
413      s.id      = id;
414      s.role     = FOLLOWER;
415      s.term     = 0;
416      s.voted_for = -1;
417
418      /* crea e lega il socket UDP */
419      s.fd = socket(AF_INET, SOCK_DGRAM, 0);
420      if (s.fd < 0) { perror("socket"); exit(1); }
421
422      struct sockaddr_in a = {
423          .sin_family   = AF_INET,
424          .sin_port     = htons(BASE_PORT + id),
425          .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
426      };
427      if (bind(s.fd, (struct sockaddr *)&a, sizeof(a)) < 0) {
428          perror("bind"); exit(1);
429      }
430      reset_election_timer(&s);
431      printf("[S%d] avviato sulla porta %d\n", id, BASE_PORT + id);
432
433      for (;;) {
434          long now = ms_now();
435          long wait;
436
437          /* calcola quanto aspettare prima del prossimo evento temporale */
438          if (s.role == LEADER)
439              wait = HB_MS - (now - s.hb_sent_at);
440          else
441              wait = s.elect_ms - (now - s.hb_recv_at);
442          if (wait < 1) wait = 1;
443
444          fd_set fds;
445          FD_ZERO(&fds);
446          FD_SET(s.fd, &fds);
447          struct timeval tv = { 0, wait * 1000L };
448
449          int ready = select(s.fd + 1, &fds, NULL, NULL, &tv);
450
451          if (ready > 0) {
452              /* messaggio in arrivo */
453              msg_t m;
454              recvfrom(s.fd, &m, sizeof(m), 0, NULL, NULL);
455
456              /* 5 .1: qualsiasi messaggio con term > currentTerm
457               * ci riporta follower immediatamente */
458              if (m.h.term > s.term)
459                  become_follower(&s, m.h.term);
460

```

```

461         switch (m.h.type) {
462             case MSG_RV:      on_request_vote(&s,      &m.rv);      break;
463             case MSG_RVR:     on_vote_response(&s,     &m.rvr);     break;
464             case MSG_AE:      on_append_entries(&s,    &m.ae);      break;
465             case MSG_AER:     on_append_response(&s,   &m.aer);     break;
466             case MSG_CLIENT:  on_client_cmd(&s,        &m.client);  break;
467         }
468     } else {
469         /* timeout: nessun messaggio ricevuto entro il tempo atteso */
470         now = ms_now();
471         if (s.role == LEADER) {
472             if (now - s.hb_sent_at >= HB_MS)
473                 send_heartbeats(&s);
474         } else {
475             /* nessun heartbeat ricevuto: il leader      morto,
476                candidati */
477             if (now - s.hb_recv_at >= s.select_ms)
478                 start_election(&s);
479         }
480     }
481 }
482
483 /*
484  * Client interattivo (processo padre)
485  *
486  */
487 static void client_loop(void) {
488     int fd = socket(AF_INET, SOCK_DGRAM, 0);
489     if (fd < 0) { perror("socket(client)"); return; }
490
491     printf("\n[client] pronto.\n");
492     printf("[client] Digita un intero + Invio per inviare un comando.\n");
493     printf("[client] Il leader lo aggiunger al log e lo replicher .\n\n");
494
495     int cmd;
496     while (scanf("%d", &cmd) == 1) {
497         /* invia a tutti: solo il leader lo processer */
498         msg_client_t m = {
499             .h = { MSG_CLIENT, 0, N },
500             .cmd = cmd,
501         };
502         for (int i = 0; i < N; i++)
503             udp_send(fd, BASE_PORT + i, &m, sizeof(m));
504         printf("[client] comando %d inviato\n", cmd);
505     }
506     close(fd);
507 }
508
509 /*
510  * main: avvia N server e poi diventa il client
511  *
512  */
513 int main(void) {

```

```

514     printf("== Raft demo: %d server, porte %d-%d ==\n\n",
515            N, BASE_PORT, BASE_PORT + N - 1);
516
517     for (int i = 0; i < N; i++) {
518         pid_t pid = fork();
519         if (pid < 0) { perror("fork"); exit(1); }
520         if (pid == 0) {
521             server_loop(i); /* figlio: loop infinito */
522             exit(0);
523         }
524     }
525
526     /* il padre diventa il client interattivo */
527     sleep(1); /* lascia tempo ai server di avviarsi e eleggere un leader
528              */
529     client_loop();
530
531     /* alla fine (Ctrl-C o EOF): termina tutti i figli */
532     for (int i = 0; i < N; i++) wait(NULL);
533     return 0;
534 }

```